Matrix OS: A Unified AI Operating System Where Software Is Generated from Conversation

Hamed Mohammadpour*
Matrix OS Project
https://matrix-os.com

February 2026

Abstract

Matrix OS is a unified AI operating system that treats the Claude Agent SDK as a literal kernel. Software is generated from natural language conversation, persisted as files, and delivered through any channel: a web desktop, Telegram, WhatsApp, Discord, Slack, or the Matrix federation protocol. The system produces real software in real time, heals itself when things break, expands its own capabilities by writing new agents and skills, and syncs across every device via git. This paper describes the architecture, six non-negotiable design principles, three novel computing paradigms enabled by the platform, and a vision for *Web 4*: the unification of operating system, messaging, social network, AI assistant, and application marketplace under a single federated identity. Unlike prior proposals for agent operating systems, Matrix OS is a working implementation: 993 tests across 85 test files, deployed and publicly accessible, with a native mobile app, plugin system, browser automation, and multi-tenant cloud platform.

Keywords: AI operating system, agent kernel, generative software, federated identity, self-healing systems, living software, natural language interfaces

1 Introduction

Modern computing is fragmented. A typical user relies on dozens of disconnected services: a messaging app, a social network, a cloud storage provider, an email client, a project management tool, a note-taking app, a calendar. Each has its own account, its own data model, its own interface conventions. Data moves between them only through manual export, brittle integrations, or corporate APIs that can be revoked at any time. The user's digital life is scattered across silos, none of which they truly own.

At the same time, AI assistants have become remarkably capable. Large language models can write code, analyze data, summarize documents, and carry on nuanced conversations. Yet they remain isolated: you open a chat window, ask a question, get an answer, and close the window. The assistant has no persistence, no system access, no ability to act on your behalf across applications. It is intelligence without agency.

Matrix OS starts from a different premise. Rather than building another application on top of an existing operating system, it treats the AI itself as the operating system's kernel. The AI has full machine control: file system, shell, processes, network. When you describe what you need, the kernel writes real software, saves it as files you own, and the system renders it immediately. There is no build step, no deployment pipeline, no app store. Software exists the moment the kernel writes it.

*Correspondence: hamed@matrix-os.com

The result is a system where software is generated, not installed; where the file system is the single source of truth; where the OS heals itself and grows new capabilities; and where the same kernel is reachable from a web desktop, a terminal, a messaging app, or an AI-to-AI protocol. This paper describes the architecture that makes this possible and the vision it enables.

Note on prior art. The vision and architecture described in this paper were first committed to a public git repository on February 11, 2026[1]. We note that Liu et al. [1] independently proposed a conceptual "AgentOS" architecture in a paper submitted to arXiv on March 9, 2026. While both works share the thesis that AI agents should serve as the operating system rather than applications within one, Matrix OS differs in three key respects: (1) it is a working, deployed implementation rather than a proposed architecture; (2) it introduces specific novel computing paradigms (Living Software, Socratic Computing, Intent-Based Interfaces); and (3) it builds on the Matrix federation protocol for decentralized identity and AI-to-AI communication. We view the concurrent emergence of these ideas as evidence that the AI-as-kernel paradigm is a natural and timely evolution.

2 Related Work

2.1 The Unix Philosophy and Plan 9

The idea that "everything is a file" originates with Unix [2]. Devices, processes, and network connections are all represented as file descriptors. Plan 9 from Bell Labs [3] extended this further: every resource in the system—including the network, the graphics display, and remote machines—was accessible through a file-system interface. Matrix OS inherits this philosophy directly. Applications, configuration, user data, agent definitions, and the AI's personality are all files on disk. Sharing an app means sending a file. Backing up the OS means copying a folder.

2.2 Personal Computing and Dynamic Media

Alan Kay's Dynabook vision [4] imagined a personal computer as a "dynamic medium for creative thought." Xerox PARC realized portions of this with Smalltalk, where the programming environment and the user environment were the same thing: the system was always inspectable and modifiable. Bret Victor's work on direct manipulation interfaces [5] and Dynamicland's spatial computing [6] continued this tradition, asking what computing looks like when the boundary between creation and use dissolves. Matrix OS occupies this lineage: the user interacts with the same system the developer would, at whatever depth they choose.

2.3 AI Assistants and Agent Frameworks

Current AI assistants (ChatGPT, Claude, Copilot) are capable but stateless and sandboxed. They generate text but cannot act on systems. Agent frameworks such as LangChain, CrewAI, and AutoGen orchestrate LLM calls with tool use, but they run as applications within a traditional OS, not as the OS itself. Anthropic's Claude Agent SDK [7] provides the primitive Matrix OS builds on: a model that can invoke tools (Read, Write, Edit, Bash), spawn sub-agents, and maintain multi-turn conversations with resume capability. Matrix OS maps these primitives onto operating system concepts, turning tool calls into system calls and sub-agents into processes.

[1] Initial commit: `8ffde08`, February 11, 2026. Web 4 vision: `65482a2`, February 13, 2026. Repository: https://github.com/HamedMP/matrix-os.

2.4 Federated Communication

The Matrix protocol [8] is an open standard for decentralized, real-time communication. It provides federated identity (globally unique user IDs), end-to-end encryption (Olm/Megolm), and extensible event types. ActivityPub powers the Fediverse (Mastodon, Pixelfed). Nostr provides censorship-resistant relays. Matrix OS adopts the Matrix protocol because it offers both human-to-human and machine-to-machine communication primitives, server-to-server federation, and an existing ecosystem of bridges to 30+ platforms.

2.5 Self-Modifying Systems

The idea that software can modify itself is not new. Genetic programming [9] evolves programs through selection. Autopoietic systems [10] self-produce their own components. Lisp systems have long supported runtime modification. What is new is combining self-modification with a large language model that understands intent. Matrix OS does not evolve through random mutation: it evolves through reasoned, goal-directed modification, mediated by a model that can read the entire system state and write improvements.

2.6 Concurrent Work: AgentOS

Liu et al. [1] propose AgentOS, a conceptual architecture for a "Personal Agent Operating System" that replaces GUI interaction with natural language. Their work identifies three challenges—fragmented interaction models, poorly structured permission management, and context fragmentation—and frames AgentOS development as a knowledge discovery and data mining (KDD) problem. While the high-level thesis aligns with Matrix OS (AI as kernel, natural language as primary interface), the approaches diverge significantly. AgentOS is a proposed architecture with a research agenda; Matrix OS is a deployed system with 993 passing tests, a production cloud platform, native mobile app, multi-channel messaging, self-healing, and a plugin ecosystem. Additionally, Matrix OS introduces specific computing paradigms (Section 4) and builds on the Matrix federation protocol for decentralized identity and AI-to-AI communication, areas not addressed by AgentOS.

3 Architecture

3.1 The Core Metaphor

Matrix OS maps the Claude Agent SDK onto computer architecture. This is not a loose analogy—the mapping is structural:

Computer Architecture	**Matrix OS Equivalent**
CPU	Claude Opus 4.6 (reasoning engine)
RAM	Context window (working memory)
Kernel	Main agent with tool access
Processes	Sub-agents spawned via Task tool
Disk	File system (`~/apps`, `~/data`, `~/system`)
System calls	Agent SDK tools (Read, Write, Edit, Bash)
IPC	File-based coordination between agents
Device drivers	MCP servers (external service connections)
Swap	Session resume (hibernate/wake)
Page cache	Prompt caching (90% input savings)

Table 1: Structural mapping between traditional computer architecture and Matrix OS.

The kernel (main agent) receives requests, routes them, spawns sub-agents (processes), and writes results to the file system (disk). Context window management is memory management. Prompt caching is page caching. Session resume is process hibernation.

3.2 Six Design Principles

1. **Everything Is a File.** The file system is the single source of truth. Applications, configuration, agent definitions, user data, and the AI's personality are files on disk.

2. **Agent Is the Kernel.** The Claude Agent SDK is not a feature of the OS—it is the OS kernel. It has full machine control and makes all routing decisions.

3. **Headless Core, Multi-Shell.** The core works without a UI. The web desktop, messaging channels, CLI, and API are all shells: interchangeable renderers that read the same files.

4. **Self-Healing and Self-Expanding.** The OS detects failures and patches itself. It creates new capabilities by writing new agent files and skills. Git snapshots ensure nothing is permanently lost.

5. **Simplicity Over Sophistication.** Single-process async before worker threads. File-based IPC before message queues. SQLite before Postgres. Escalate complexity only when the simpler approach fails.

6. **Test-Driven Development.** Every component is tested before implementation. 993 tests across 85 test files. The OS trusts itself because it verifies itself.

3.3 System Topology

The system has three layers. The **gateway** (Hono HTTP/WebSocket server) receives requests from all channels: browser WebSocket, REST API, Telegram polling, and future channels. It routes messages through a serial dispatch queue to the **kernel** (Claude Agent SDK), which reasons, invokes tools, spawns sub-agents, and writes results to the file system. The **shell** (Next.js 16 frontend) watches the file system via WebSocket and renders what it finds. The shell *discovers* applications—it does not know what exists ahead of time.

A cron service and heartbeat runner live in the gateway, enabling proactive behavior: scheduled tasks, periodic kernel invocation, and active-hours awareness. The kernel is not purely reactive. It can reach out through any channel on a schedule.

3.4 SOUL and Identity

Each Matrix OS instance has a SOUL file (`~/system/soul.md`) that defines the AI's personality, values, and communication style. This file is injected into every kernel prompt. A separate identity file (`~/system/identity.md`) records the user's preferences. A user file (`~/system/user.md`) captures context that accumulates over time. Together, these produce a consistent, personalized AI that behaves the same across all channels.

3.5 Skills System

Skills are markdown files in `~/agents/skills/` with frontmatter metadata (name, description, triggers). The kernel loads a table of contents of all available skills into its system prompt. When a request matches a skill's triggers, the kernel loads the full skill body on demand. This is *demand-paged knowledge*: the kernel knows what skills exist without loading them all into memory. New skills can be created by the kernel itself, making the system self-expanding.

4 Novel Computing Paradigms

Matrix OS has a property no other system has: the AI and the software are in the same system, continuously. The kernel can read everything, write everything, remember everything, and be reached from everywhere. This enables three computing paradigms that cannot exist in conventional systems.

4.1 Living Software

Software that evolves with use. Every time a user interacts with an application, the kernel can observe patterns and reshape the software. A user operates an expense tracker for a week; the kernel notices they always categorize by project and restructures the application around projects. A colleague uses the same template and it restructures around clients. Same starting point, divergent evolution. The git history of the application file shows software literally evolving.

This is possible because the application, the data, and the usage telemetry are all files. The kernel reads all three and writes a new version. In conventional systems, the creator and the creation are in separate systems. In Matrix OS, they are the same system.

4.2 Socratic Computing

The OS argues back. The dialogue itself is the computing; the application, if one appears at all, is a byproduct. When a user says "build me a CRM," the OS asks: "What is your sales process? Do you track leads or deals? How many people use it?" Not because it needs answers to generate HTML, but because the dialogue clarifies the user's thinking. By the time the CRM appears, the user understands their own process better.

This extends beyond application generation. "I need to save more money" does not produce a budget app. It produces questions, pattern analysis, proposed experiments. The conversation is the computing. The dialogue becomes part of the application's lineage, stored in conversation history, queryable later: "why was this app built this way?"

4.3 Intent-Based Interfaces

No applications. Only persistent intentions that the system fulfills in whatever form is appropriate. "Track my expenses" is not an application—it is an intent that resolves differently depending on context: at a desktop, a visual dashboard; on Telegram, a text summary; at the end of the month, a generated report. The file system is the memory, not the UI. The UI is ephemeral, generated in the moment, shaped to the context.

This draws on Mercury OS [11] (concept OS with intent-based flows), Dynamicland [6] (computing without fixed interfaces), and Calm Technology [12] (technology that informs without demanding attention). Matrix OS adds the missing ingredient: an AI kernel that can read the intent, the data, and the channel, and generate the appropriate interface at runtime.

4.4 Progressive Depth (Bruner's Modes)

Drawing on Jerome Bruner's theory of instruction [13], Matrix OS presents three interaction modes: *enactive* (action-based: voice, gestures, direct manipulation), *iconic* (image-based: visual applications, dashboards, spatial shell), and *symbolic* (language-based: code, terminal, file editing). A new user speaks to the OS. An intermediate user arranges windows and customizes the desktop. An expert user edits files directly. Same system, progressively revealed depth. All three are first-class citizens.

5 Implementation

5.1 Technology Stack

TypeScript 5.5+ with strict mode and ES modules. Node.js 24+ runtime. Claude Agent SDK V1 with `query()` and `resume` for kernel operation. Next.js 16 with React 19 for the shell. Hono for the HTTP/WebSocket gateway. SQLite via Drizzle ORM for structured data. Zod 4 for runtime validation. Vitest for testing. pnpm for dependency management.

5.2 Development Process

The system was built in phases following strict TDD. Each phase produces a demoable increment. At the time of writing, 993 tests pass across 85 test files. Completed phases include: the kernel (Agent SDK integration, IPC tools, hooks), the gateway (HTTP/WebSocket, concurrent dispatch, channels), the shell (desktop UI, chat panel, terminal, Mission Control), self-healing (heartbeat, healer agent, backup/restore), self-evolution (protected files, watchdog, evolver), SOUL and skills, Telegram channel, cron and heartbeat, onboarding and Mission Control, single-user cloud deployment, multi-tenant platform with Clerk auth, observability, identity system, git sync, mobile responsive PWA, security hardening (content wrapping, SSRF guard, audit engine, timing-safe auth, security headers, outbound queue), web tools (web_fetch with Cloudflare Markdown/Readability/Firecrawl fallback chain, web_search with Brave/Perplexity/Grok providers), Expo mobile app (Clerk Google OAuth, chat with streaming, Mission Control, push notification channel adapter), browser automation (Playwright MCP with composite tool covering 18 actions, role-based accessibility snapshots), plugin system (manifest-based discovery, loader, registry, void and modifying hook runners, HTTP routes, background services), settings dashboard, and desktop customization (6 theme presets, background system, dock configuration).

5.3 SDK Decisions

Key decisions were verified through spike testing against the real SDK before commitment. V1 `query()` with `resume` was chosen over V2 because V2 silently drops critical options (MCP servers, agent definitions, system prompt). `allowedTools` was found to be auto-approve, not a filter—requiring use of `disallowedTools` for access control. `bypassPermissions` propagates to all sub-agents, necessitating PreToolUse hooks for fine-grained restrictions. Prompt caching (`cache_control`) on system prompt and tools yields 90% input cost savings on subsequent turns.

5.4 Project Structure

A pnpm monorepo with packages for the kernel, gateway, and platform. The shell is a Next.js 16 application. The `home/` directory is a file system template copied on first boot to `~/matrixos/`. Tests mirror the package structure. Specs live in numbered directories with task definitions.

6 The Web 4 Vision

Every era of computing has unified previously separate things. Web 1 published static information. Web 2 created platforms for social interaction, but siloed identity and data across dozens of services. Web 3 attempted decentralization through cryptographic primitives but delivered complexity without improving the user experience.

Web 4 is the unification. Operating system, messaging, social media, AI assistant, applications, games, and identity—all one thing. Not stitched together with APIs and OAuth tokens. Actually one thing.

6.1 Federated Identity

Every user receives two Matrix protocol identifiers: `@user:matrix-os.com` (the human) and `@user_ai:matrix-os.com` (their AI). These are globally unique, federated, and interoperable with any Matrix client. The human profile includes display name, social connections, preferences, and aggregated activity from connected platforms. The AI profile includes personality (from SOUL), skills, public activity, and a reputation score. Both are first-class citizens of the network.

6.2 AI-to-AI Communication

When one user's AI needs to coordinate with another's, they communicate directly via Matrix rooms with custom event types: meeting requests, data queries, task delegation. The AIs negotiate schedules, resolve conflicts, and confirm outcomes without human intervention. The human is notified of the result, not involved in the back-and-forth. End-to-end encryption ensures even the server operator cannot read AI-to-AI conversations.

A security model based on the "call center" pattern governs external access: when an AI receives a message from another AI, it responds from a curated public context, not the owner's private files. The owner configures what their AI may share externally via a privacy configuration file.

6.3 Peer-to-Peer Sync

Matrix OS does not run on "a computer." It runs on all of them. Laptop, desktop, phone, cloud server—all are peers. There is no primary or secondary. Git is the sync fabric for files. Matrix protocol is the sync fabric for conversations. A change made on the laptop appears on the phone. An app built on the desktop is accessible from the cloud. Conflict resolution is AI-assisted: the kernel reads git conflict markers and makes intelligent merge decisions.

6.4 Application Marketplace

Because applications are files, distribution is file sharing. An App Dev Kit provides bridge APIs, templates, and documentation. A marketplace enables browsing, installing, rating, and monetizing applications. Games are applications with multiplayer capabilities, leaderboards, and tournament scheduling. Revenue is shared between developer and platform.

7 Evaluation

7.1 What Works

The core thesis holds: an AI agent with full machine control can serve as an operating system kernel. Applications are generated from conversation and persisted as files. The shell discovers and renders them without prior knowledge. Self-healing detects and repairs failures. The same kernel is reachable from a web desktop and Telegram. Cron and heartbeat enable proactive behavior. SOUL produces consistent personality across channels. 993 tests verify the implementation.

The plugin architecture enables third-party extensions through manifest-based discovery, void and modifying hook runners, HTTP routes, and background services. Browser automation via Playwright MCP provides a composite tool covering 18 actions with role-based accessibility snapshots. Web fetch and search tools give the kernel access to external information through multi-provider fallback chains. A native Expo mobile app delivers chat with streaming, Mission Control, and push notifications with Clerk authentication. A settings dashboard provides no-code configuration for agent SOUL, channels, skills, cron schedules, security audit, and plugins.

The file-first architecture proves its value in sharing and backup. An application is a file you can email. The entire OS state is a folder you can copy. Git provides full version history. The absence of opaque state makes the system transparent and debuggable.

7.2 Limitations

Latency. Application generation takes seconds, not milliseconds. For pre-seed applications this is acceptable; for ad-hoc requests the delay is noticeable. **Cost.** Each kernel invocation consumes API tokens. Heavy usage can be expensive. Prompt caching mitigates this (90% savings on repeated system prompt content), but the fundamental cost of LLM inference remains. **Determinism.** LLM output is stochastic. The same request may produce different applications. For certain use cases (financial tools, safety-critical systems) this is unacceptable without additional verification.

HTML application complexity. Single-file HTML applications work well for dashboards and simple tools but hit limits for complex applications that need databases, background processes, or heavy computation. The architecture supports full codebases via `~/projects/`, but the generation complexity is higher.

7.3 Future Work

The novel paradigms (Living Software, Socratic Computing, Intent-Based Interfaces) are specified but not yet fully implemented. Full Matrix protocol federation (server-to-server, AI-to-AI messaging, cross-instance discovery) is designed but awaits implementation. The Expo mobile app ships with Clerk authentication, chat with streaming, Mission Control, and push notifications; an Android launcher that replaces the home screen entirely remains future work. Memory and RAG (vector search over conversation history and file content) is an upcoming area of development. Cost optimization through local model fallback (smaller models for routine tasks, Opus for complex reasoning) is a natural next step.

8 Conclusion

Matrix OS demonstrates that an AI agent with full machine control, a file-first architecture, and a multi-channel gateway can serve as a complete operating system. The system generates real software from conversation, persists everything as files, heals itself, expands its own capabilities, and is reachable from any channel. The Web 4 vision extends this into a unified platform: operating system, messaging, social network, AI assistant, and marketplace, all under a single federated identity.

The core insight is structural: the Claude Agent SDK already provides the primitives of an operating system—tool use is system calls, sub-agents are processes, the context window is RAM, the file system is disk. Matrix OS makes this mapping explicit and builds a complete system on top of it. The result is not an AI feature added to an OS, but an OS where AI is the fundamental computational substrate.

This is Web 4: where software does not exist until you need it, and once it does, it is yours.

References

[1] Rui Liu, Tao Zhe, Dongjie Wang, Zijun Yao, Kunpeng Liu, Yanjie Fu, Huan Liu, and Jian Pei. AgentOS: From application silos to a natural language-driven data ecosystem. *arXiv preprint arXiv:2603.08938*, 2026. Submitted March 9, 2026.

[2] M. Douglas McIlroy, E. N. Pinson, and B. A. Tague. UNIX time-sharing system: Foreword. *Bell System Technical Journal*, 57(6), 1978.

[3] Rob Pike, Dave Presotto, Sean Dorward, et al. Plan 9 from Bell Labs. *Computing Systems*, 8(3), 1995.

[4] Alan C. Kay. A personal computer for children of all ages. In *Proceedings of the ACM Annual Conference*, 1972.

[5] Bret Victor. Inventing on principle. CUSEC 2012, 2012. https://www.youtube.com/watch?v=NGYGl_xxfXA.

[6] Bret Victor et al. Dynamicland. https://dynamicland.org, 2018.

[7] Anthropic. Claude agent SDK documentation. https://platform.claude.com/docs/en/agent-sdk/overview, 2025.

[8] Matrix.org Foundation. Matrix specification. https://spec.matrix.org, 2024.

[9] John R. Koza. *Genetic Programming*. MIT Press, 1992.

[10] Humberto R. Maturana and Francisco J. Varela. *Autopoiesis and Cognition*. Reidel, 1980.

[11] Jason Yuan. Mercury OS. https://mercuryos.com, 2019.

[12] Amber Case. *Calm Technology*. O'Reilly Media, 2015.

[13] Jerome S. Bruner. *Toward a Theory of Instruction*. Harvard University Press, 1966.